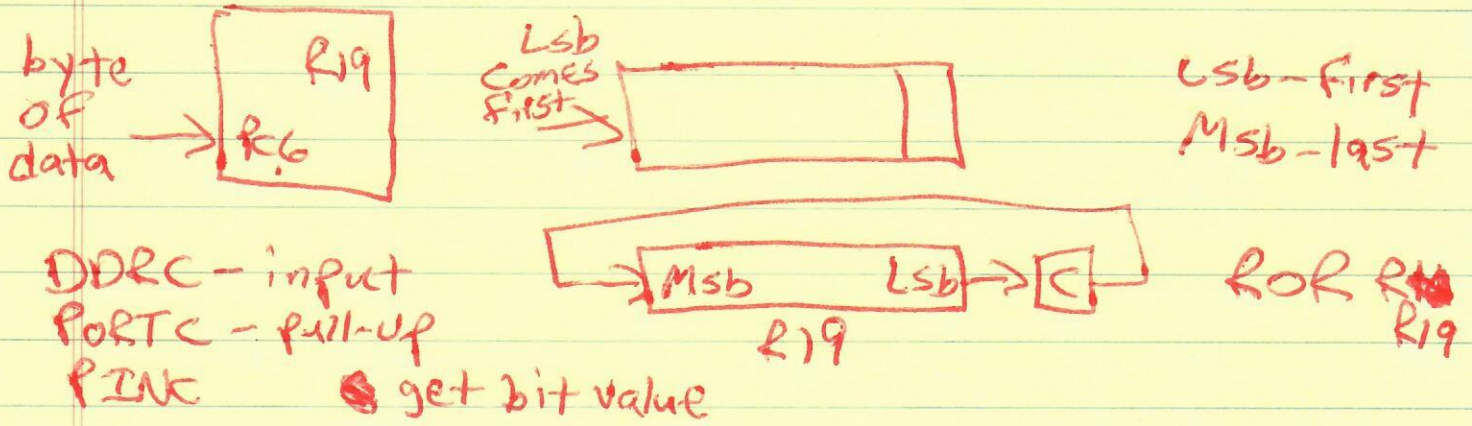
Shift and Rotate Operations

LSL Rd; logical shift left - most significant bit shifted to C, least significant bit gets a 0
LSR Rd; logical shift right - least significant bit shifted to C, most significant bit gets a 0
ROL Rd; rotate left-current C goes to least signif. bit, most signif. bit to C
ROR Rd; rotate right-current C goes to most signif. bit, least signif. bit to C

Write a program to bring in a byte of data serially via PC6 and save it to R19. The byte comes in with the least significant bit first.

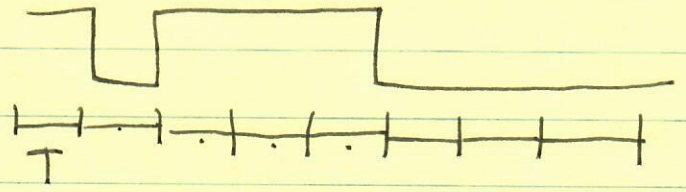



```
        .ORG 0
        CBI  DDRC,6  ; PC6 input   0 = input
        SBI  PORTC,6 ; pull-up resistor enable
        LDI  R19, 0  ; R19 = 0x00
        LDI  R17,8   ; 8 bits of data

AGAIN:  SBIC PINC,6  ; PINC.6 = 0?
        RJMP set_C   ;  PINC.6 = 1
        CLC          ;  C=0 ⇒ PINC.6 = 0
        RJMP Nxt     ; go to rotation

set_C:  SEC          ;  C=1
Nxt:    ROR R19      ; rot right R19
                       move C to Msb of R19
        DEC R17      ; decrement bit count
        BRNE AGAIN
Here:   RJMP HERE    ; terminal loop
```

port bit config initialization


T

BCD & ASCII Conversion

0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

Unpacked BCD

- lower 4 bits represents the BCD value

0x04 ⇒ ~~0x00~~ 0100
4

packed BCD

byte — upper nibble (BCD) | lower nibble (BCD)

0x47 ⇒ 0100 0111

Real Time clock ⇒ sec, time, day, month, year

SWAP Rd ; swap nibbles

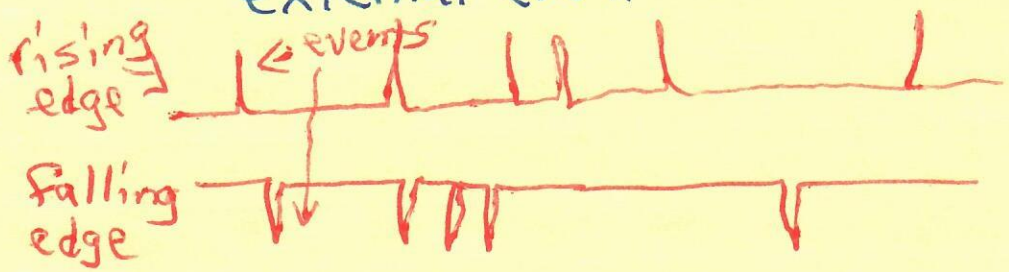
```
LDI R16, 0x48
SWAP R16 ; R16= 0x84
```

Chapter 9 ⇒ timer/counter

Microcontroller applications commonly involve
① detecting/counting events and ② generating periods of time or time delays

① → asynchronous
② → synchronous

events ⇒ requires an external clock source
⇒ counter <u>increments</u> with each external event



periods of time or time delays
⇒ connecting oscillator to clock pin for counter
↳ ⇒ $\frac{1 CC}{1 MC}$ ⇒ AVR ⇒ Atmega32
↳ ⇒ counter <u>increments</u> once every machine cycle (MC) ⇒ increments once every clock cyc

if we want a delay of 50μs
$f_{osc} = 1 MHz \Rightarrow t_{MC} = \left(\frac{1 CC}{1 MC}\right)\left(\frac{1 s}{10^6 CC}\right) = 1 \mu s/MC$
then we need a counter of 50

50 ⇒ find 50 in binary

⇒ 00110010

take 2s comp

11001101

\+ 1

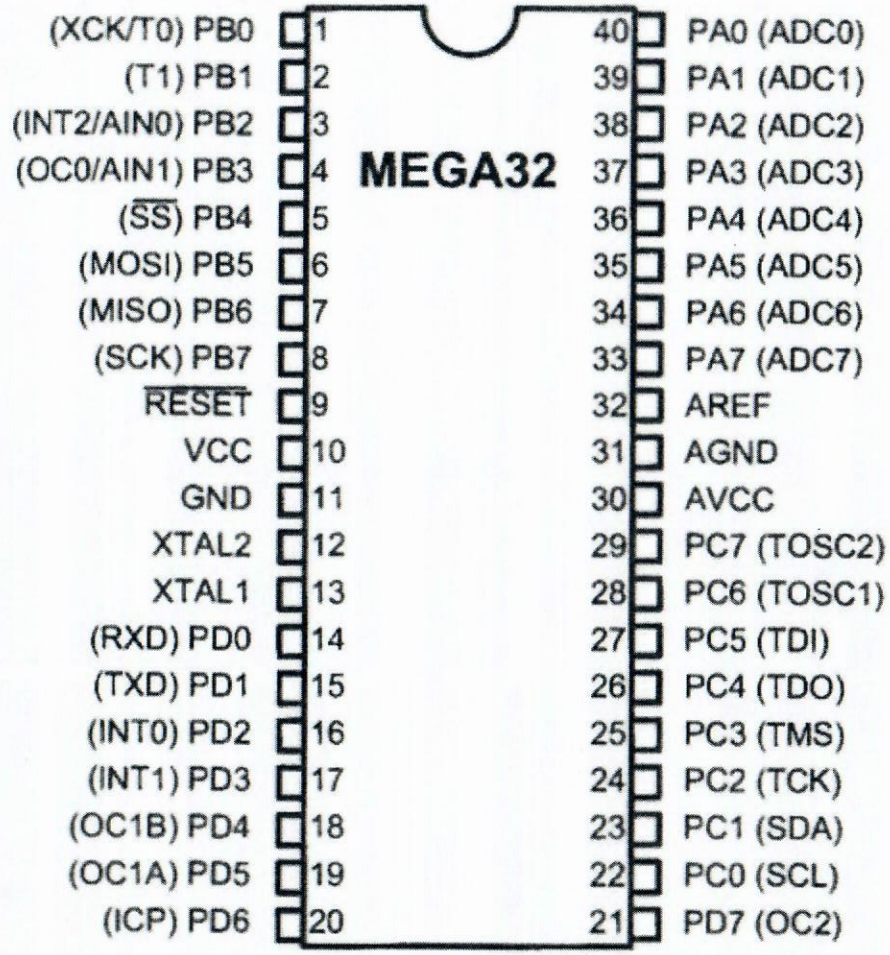
counter ⇒ 1100 1110

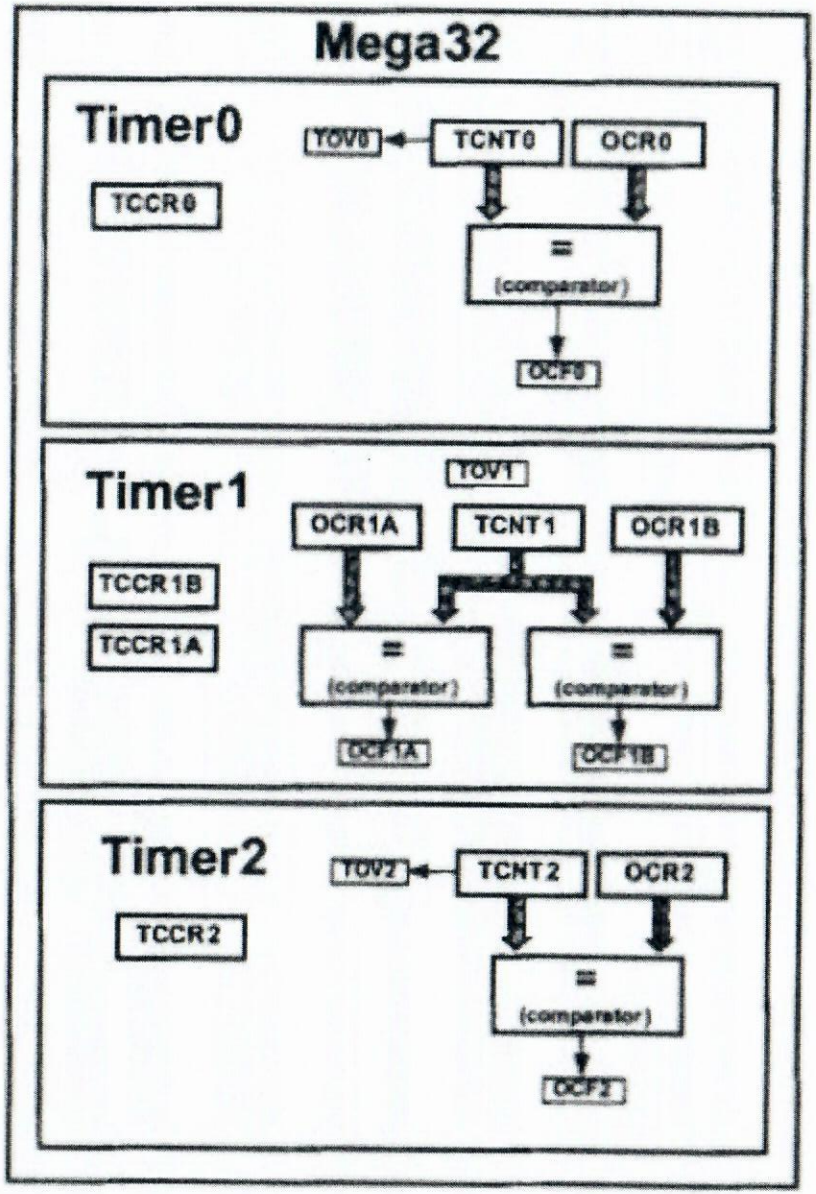
C E

0xCE → 0xCF → 0xD0 → . . .

→ 0xFF → 0x00

Chapter 4-Fig4-1




Mega32
Timer0
TOV0
TCNT0
OCR0
TCCR0
=
(comparator)
OCF0
Timer1
TOV1
OCR1A
TCNT1
OCR1B
TCCR1B
TCCR1A
=
(comparator)
=
(comparator)
OCF1A
OCF1B
Timer2
TOV2
TCNT2
OCR2
TCCR2
=
(comparator)
OCF2

Timer/Counter Registers:

① TCNTn ↑ increments
TCNT0 TCNTn = 0
TCNT1 (a b) load TCNTn
TCNT2

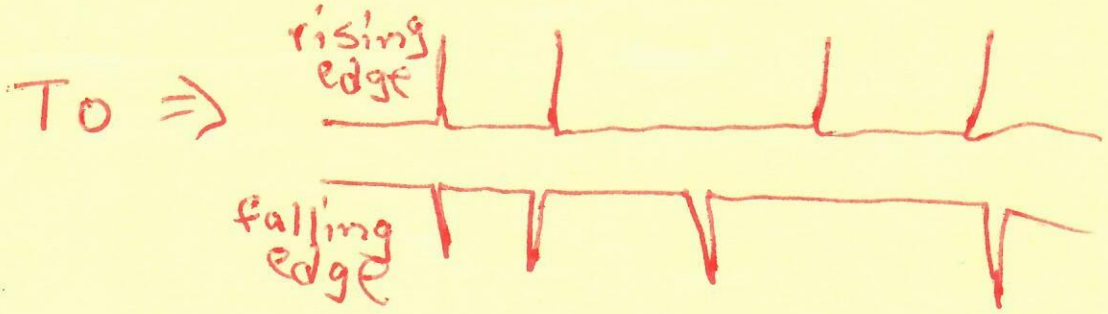
OUT I/O Registers

② TOVn (timer/counter overflow) (TIFR)
TIFR ⇒ timer interrupt flag register

TOV0
TOV1 TOVn = 0 if no overflow
TOV2 TOVn = 1 if overflow

resetting flag write a 1 to TOVn & 0s to other bits of TIFR

③ TCCRn (Timer/counter control Register)



④ OCRn (output Compare register)
OCR0 = 0x23 ⇒ When TCNT0 = 0x23
OCFn (output compare match flag) ⇒ TCNT0 = 0 (clear)
⇒ OCF0 = 1



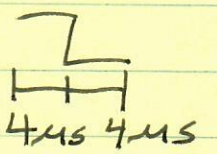
CS02:CS00

001 ⇒ F_1 ⇒ clk (no prescaling)

$$F_8 = \frac{1MHz}{8} \Rightarrow T = \frac{8}{10^6} = 8\mu s$$

CS02:CS00

010



Create a square wave of 50% duty cycle on PB.5 (PortB, bit5). $f_{osc} = 8$ MHz

```
.ORG 0
LDI R16, HIGH(RAMEND)
OUT SPH, R16
LDI R16, LOW(RAMEND)
OUT SPL, R16  ; move SP when using
                subroutines

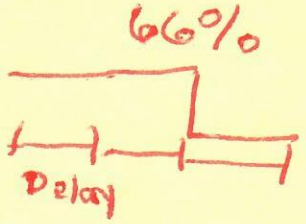
SBI DDRB, 5 ;  PORTB.5 ⇒ output
LDI R16, 0x20 ; 0b 00 1 0000
                   5 4 3 2 1 0
                   2     0

LDI R17, 0
OUT PORTB, R17 ; PORTB = 0x00 ⇒ init PB
Begin: CALL Delay
PB.5   EOR R17, R16 ;
OUT PORTB, R17
RJMP Begin
```

x	y	x⊕y	
0	0	0	
0	1	1 ←	toggle PB.5
1	0	1	
1	1	0 ←	

```
SBI PORTB, 5
call Delay
SBI PORTB, 5
call Delay
CBI PORTB, 5
call Delay
```

⇒



AVR
ICC
IMC

normal
COM01:COM00

Delay: LDI R20, 194;
OUT TCNT0, R20; TCNT0 = 194
[redacted], [redacted]; 00 000001
CLK
WGM01:WGM00
normal
LDI R20, 0x01;
OUT TCCR0, R20;

timer 0
normal mode
internal clock
no prescaler

FOC0 (0) for [redacted] our purposes

TCNT0 → 0xFF → 0x00

Again: IN R20, TIFR; TIFR ⇒ TOV0 flag
SBRS R20, TOV0; if TOV0=1, skip ⇒ timer overflow
RJMP Again;
TOV0=0
LDI R20, 0x00;
OUT TCCR0, R20; turn off timer0 (CS02:CS00 000)
LDI R20, (1<<TOV0); 00000001
OUT TIFR, R20; clear TOV0 flag write 1 to TOV0 0s to other bits
RET

$f_{osc} = 8\,MHz \Rightarrow T_{MC} = \left(\frac{1cc}{1MC}\right)\left(\frac{1s}{8\times10^6\,cc}\right)$

$T_{MC} = 0.125\,\mu s/MC$

$\frac{1cc}{1MC}$

194
preload

194 preload

$$\begin{array}{r} 256 \\ -\,194 \\ \hline 62 \end{array}$$

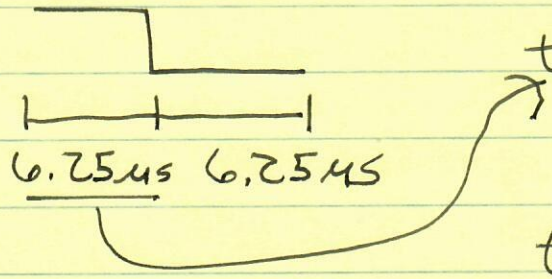
-62
$C2$

194 → hex

LDI R20, -62
OUT TCNT0, R20

Assume $f_{osc} = 8\,MHz$. Write a program to generate an 80kHz square wave on PORTC bit 2 (RC2). Timer 0

$f = 80kHz \Rightarrow T = \frac{1}{f} = \frac{1}{80(10^3)} = 12.5\mu s$



$t_{delay} = n t_{MC}$

$t_{MC} = \left(\frac{1\,cc}{TMc}\right)\left(\frac{1s}{8(10^6)\,cc}\right)$

$= 0.125\mu s/MC$

$n = \frac{6.25\mu s}{0.125\mu s} = 50$ ticks

TCNTO $= -50$ or $256 - 50 = 206$

hex

CE

```
;  in class example - 80 kHz square wave


.ORG 0
LDI R16, HIGH(RAMEND)
OUT SPH, R16
LDI R16, LOW(RAMEND)
OUT SPL, R16; MOVE SP BECAUSE USING SUBROUTINE

SBI DDRC, 2; PC2 OUTPUT
LDI R17, 0X04; 0B00000100 BIT POSITION 2 SET
LDI R18, 0
OUT PORTC, R18; INITIALIZE PC2 = 0

BEGIN:      RCALL DELAY
            EOR R18, R17; TOGGLE PC2
            OUT PORTC, R18; UPDATE PC2
            RJMP BEGIN

            ; TIMER 0 DELAY
DELAY:      LDI R21, -50; PRELOAD FOR NUMBER OF TICKS FOR HALF WAVE
            OUT TCNT0, R21
            LDI R21, 0X01; 0B00000001-LAST 3 BITS 001=> CLK NO PRESCALER
            OUT TCCR0, R21
AGAIN:      IN R21, TIFR
            SBRS R21, TOV0; CHECK STATUS OF TOV0
            RJMP AGAIN; WAIT FOR TIMER0 TO OVERFLOW (TOV0 SET)
            LDI R21, 0X00
            OUT TCCR0, R21; TURN OFF TIMER 0
            LDI R21, (1<<TOV0); RESET TOV0 (WRITE A 1 TO IT, 0S TO OTHER BITS)
            OUT TIFR, R21
            RET
```

Bit	7	6	5	4	3	2	1	0
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00
Read/Write	W	RW	RW	RW	RW	RW	RW	RW
Initial Value	0	0	0	0	0	0	0	0

FOC0 D7 Force Output compare: Writing 1 to it forces the wave generator to act as if a compare match has occurred.

WGM01:00	D6	D3	Timer0 mode selector bits
	0	0	Normal
	0	1	CTC (Clear Timer on Compare match)
	1	0	PWM, phase correct
	1	1	Fast PWM

COM01:00 D5 D4 Compare Output Mode; The table shows what the wave generator does on compare match when the timer is in Normal or CTC mode:

COM01	COM00	Description
0	0	Normal port operation, OC0 disconnected
0	1	Toggle OC0 on compare match
1	0	Clear OC0 on compare match
1	1	Set OC0 on compare match

CS02:00 D2D1D0 Timer0 clock selector

0	0	0	No clock source (Timer/Counter stopped)
0	0	1	clk (no prescaling)
0	1	0	clk / 8
0	1	1	clk / 64
1	0	0	clk / 256
1	0	1	clk / 1024
1	1	0	External clock source on T0 pin. Clock on falling edge
1	1	1	External clock source on T0 pin. Clock on rising edge

Figure 15-4. TCCR0 (Timer/Counter Control Register) Register

Bit	7	6	5	4	3	2	1	0
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00
Read/Write	W	RW	RW	RW	RW	RW	RW	RW
Initial Value	0	0	0	0	0	0	0	0

FOC0 D7 Force compare match: This is a write-only bit, which can be used while generating a wave. Writing 1 to it causes the wave generator to act as if a compare match had occurred.

WGM00, WGM01

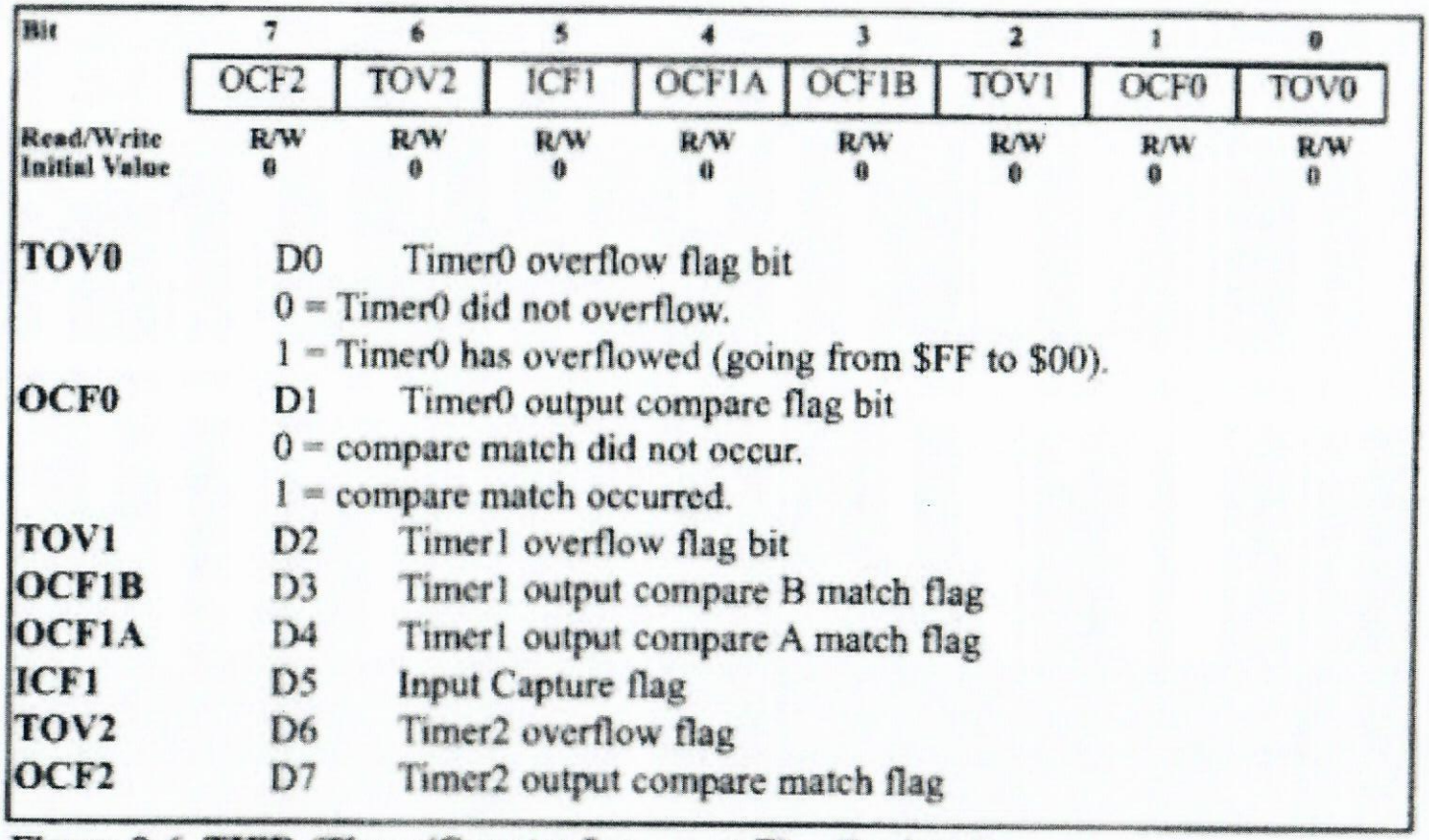
D6	D3	Timer0 mode selector bits
0	0	Normal
0	1	CTC (Clear Timer on Compare Match)
1	0	PWM, phase correct
1	1	Fast PWM

COM01:00 D5 D4 Compare Output Mode:
These bits control the waveform generator (see Chapter 15).

CS02:00

D2	D1	D0	Timer0 clock selector
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	clk (No Prescaling)
0	1	0	clk / 8
0	1	1	clk / 64
1	0	0	clk / 256
1	0	1	clk / 1024
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

Figure 9-5. TCCR0 (Timer/Counter Control Register) Register



Bit	7	6	5	4	3	2	1	0
	OCF2	TOV2	ICF1	OCF1A	OCF1B	TOV1	OCF0	TOV0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

TOV0 D0 Timer0 overflow flag bit
0 = Timer0 did not overflow.
1 = Timer0 has overflowed (going from $FF to $00).

OCF0 D1 Timer0 output compare flag bit
0 = compare match did not occur.
1 = compare match occurred.

TOV1 D2 Timer1 overflow flag bit

OCF1B D3 Timer1 output compare B match flag

OCF1A D4 Timer1 output compare A match flag

ICF1 D5 Input Capture flag

TOV2 D6 Timer2 overflow flag

OCF2 D7 Timer2 output compare match flag

Figure 9-6. TIFR (Timer/Counter Interrupt Flag Register)